

Caching

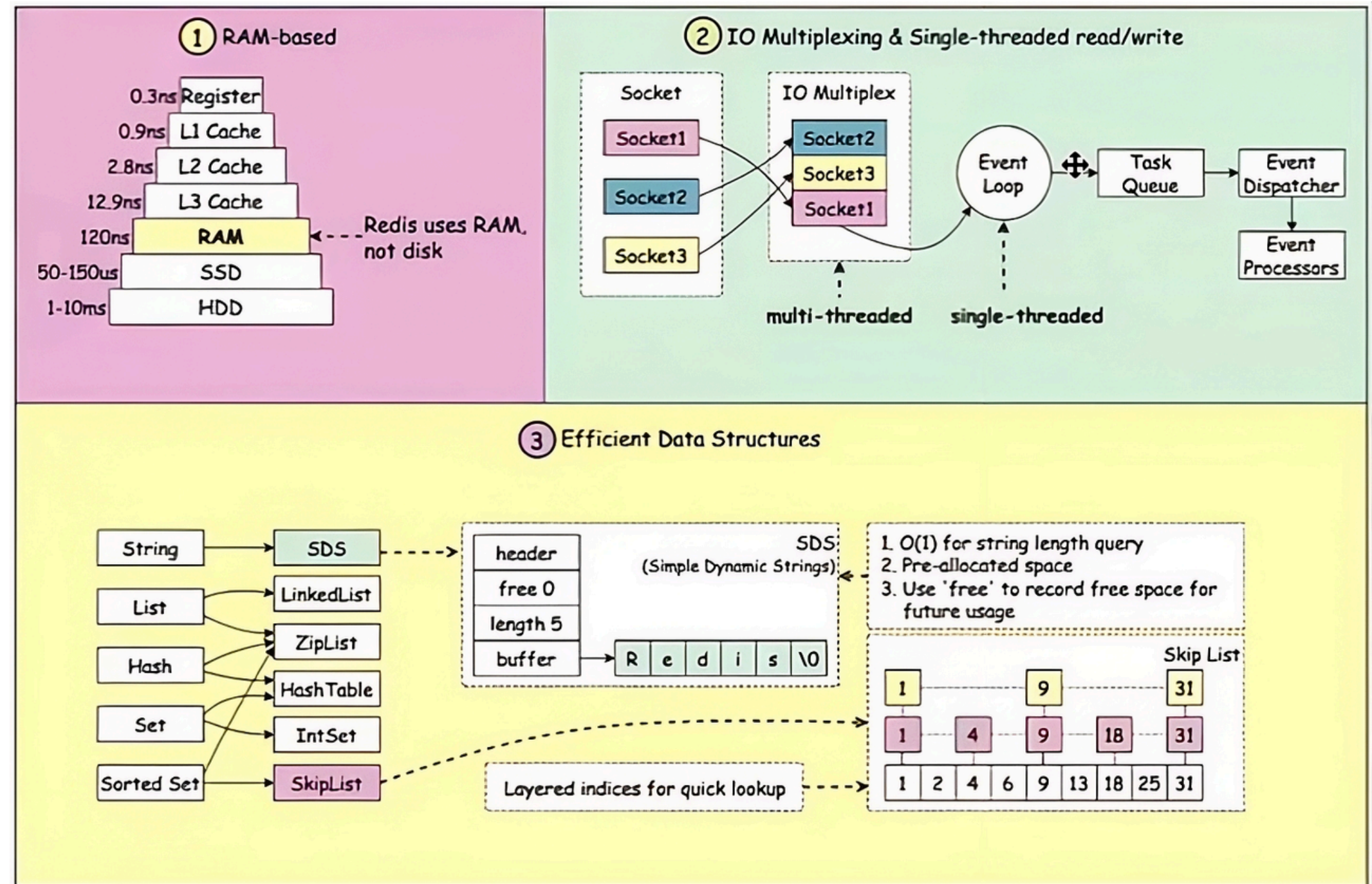
Redis

- **In-memory** database project (kind of DBMS)
- Used to store/retrieve data **faster**
- Support a lot of data structures
- Support many client libraries for various programming languages



Principles

- Data structures
- Single-threaded
- I/O multiplexing
- Support **atomic operations** on data, allowing multiple threads to access and modify data concurrently without causing conflicts.



Principles

- Key-value structure
- Key: data identifier, string
- Value: data, string, hashmap, list, set, sorted set,...
- TTL: time-to-live, to expire the data after a defined name

Principles

- Redis does not fully support **ACID** properties
 - **Atomicity:** Commands are executed atomically on a single thread.
 - **Consistency:** Full consistency is **not** guaranteed; data may be lost before persistence.
 - **Isolation:** Transactions are isolated because commands execute sequentially.
 - **Semi-Durability:** Data is stored in memory; persistence improves durability with performance trade-offs.

Commands

Strings

- Get value: **GET** <key>
- Set value: **SET** <key> <value>
- Set the value only if the key does not exist: **SETNX** <key> <value>
- Get the values of multiple keys: **MGET** <key> <key> ...
- Set the values of multiple keys: **MSET** <key> <value> <key> <value> ...

```
redis> SET bike:1 Deimos
OK
redis> GET bike:1
"Deimos"
redis>
```

Commands

Hashes

- Set field: ***HSET*** ***<key>*** ***<field>*** ***<value>***
- Get field: ***HGET*** ***<key>*** ***<field>***
- Get all fields & values: ***HGETALL*** ***<key>***
- Delete field: ***HDEL*** ***<key>*** ***<field>***
- Increment field: ***HINCRBY*** ***<key>*** ***<field>*** ***<increment>***

```
redis> HSET bike:1 model Deimos brand Ergonom type 'Enduro bikes' price 4972
(integer) 4
redis> HGET bike:1 model
"Deimos"
redis> HGET bike:1 price
"4972"
```

Commands

Lists

- Push to left: ***LPUSH*** *<key>* *<value>*
- Push to right: ***RPUSh*** *<key>* *<value>*
- Pop from left: ***LPOP*** *<key>*
- Pop from right: ***RPOP*** *<key>*
- Get range: ***LRANGE*** *<key>* *<start>* *<stop>*

```
redis> LPUSH bikes:repairs bike:1
(integer) 1
redis> LPUSH bikes:repairs bike:2
(integer) 2
redis> LMOVE bikes:repairs bikes:finished LEFT LEFT
"bike:2"
```


Commands

Sets

- Add member: ***SADD*** <key> <member>
- Remove member: ***SREM*** <key> <member>
- Get all members: ***SMEMBERS*** <key>
- Check member exists: ***SISMEMBER*** <key> <member>
- Union: ***SUNION*** <key>...

```
redis> SADD bikes:racing:france bike:1
(integer) 1
redis> SADD bikes:racing:france bike:1
(integer) 0
redis> SADD bikes:racing:france bike:2 bike:3
(integer) 2
```

Commands

Sorted Sets

- Add member: **ZADD** <key> <score> <member>
- Remove member: **ZREM** <key> <member>
- Get score: **ZSCORE** <key> <member>
- Get members by score/rank: **ZRANGE** <key> <start> <stop>
- Increment score: **ZINCRBY** <key> <inc> <member>

```
redis> ZADD racer_scores 10 "Norem" 10 "Royce" 14 "Prickett"  
(integer) 3  
redis> ZRANK racer_scores "Norem"  
(integer) 0  
redis> ZREVRANK racer_scores "Norem"  
(integer) 2
```

Commands

Bloom filter

- Create filter: ***BF.RESERVE*** *<key>* *<error_rate>* *<capacity>*
- Add item: ***BF.ADD*** *<key>* *<item>*
- Check item exists: ***BF.EXISTS*** *<key>* *<item>*

```
redis> BF.RESERVE bikes:models 0.001 1000000
OK
redis> BF.ADD bikes:models "Smoky Mountain Striker"
(integer) 1
redis> BF.EXISTS bikes:models "Smoky Mountain Striker"
(integer) 1
redis> BF.MADD bikes:models "Rocky Mountain Racer" "Cloudy City Cruiser" "Windy City Wippet"
1) (integer) 1
2) (integer) 1
3) (integer) 1
```

Commands

Count-Min Sketch

- Create sketch: ***CMS.INITBYPROB***
- Increment count: ***CMS.INCRBY***
- Query frequency: ***CMS.QUERY***

```
redis> CMS.INCRBY bikes:profit "Smokey Mountain Striker" 100
1) (integer) 100
redis> CMS.INCRBY bikes:profit "Rocky Mountain Racer" 200 "Cloudy City Cruiser" 150
1) (integer) 200
2) (integer) 150
redis> CMS.QUERY bikes:profit "Smokey Mountain Striker" "Rocky Mountain Racer" "Cloudy City Cruiser" "Terrible Bike Name"
1) (integer) 100
2) (integer) 200
3) (integer) 150
4) (integer) 0
```

Commands

HyperLogLog

- Add elements: ***PFADD*** <key> <element>...
- Estimate cardinality: ***PFCOUNT*** <key>...

```
redis> PFADD bikes Hyperion Deimos Phoebe Quaoar
(integer) 1
redis> PFCOUNT bikes
(integer) 4
redis> PFADD commuter_bikes Salacia Mimas Quaoar
(integer) 1
redis> PFMERGE all_bikes bikes commuter_bikes
OK
```


Commands

Geospatial

- Add location: ***GEOADD*** <key> <lon> <lat> <member>
- Get coordinates: ***GEOPOS*** <key> <member>
- Calculate distance: ***GEODIST*** <key> <m1> <m2>
- Search nearby: ***GEOSEARCH*** <key> ...

```
redis> GEOADD bikes:rentable -122.27652 37.805186 station:1
(integer) 1
redis> GEOADD bikes:rentable -122.2674626 37.8062344 station:2
(integer) 1
redis> GEOSEARCH bikes:rentable FROMLONLAT -122.2612767 37.7936847 BYRADIUS 5 km WITHDIST
1) 1) "station:1"
   2) "1.8523"
2) 1) "station:2"
   2) "1.4979"
```

Commands

Streams

- Add message: ***XADD*** ***<stream>*** * ***<field>*** ***<value>***
- Read messages: ***XREAD***
- Create consumer group: ***XGROUP CREATE***
- Read as consumer: ***XREADGROUP***
- Acknowledge message: ***XACK***

```
> XADD race:france * rider Castilla speed 30.2 position 1 location_id 1
"1692632086370-0"
> XADD race:france * rider Norem speed 28.8 position 3 location_id 1
"1692632094485-0"
> XADD race:france * rider Prickett speed 29.7 position 2 location_id 1
"1692632102976-0"
```

Commands

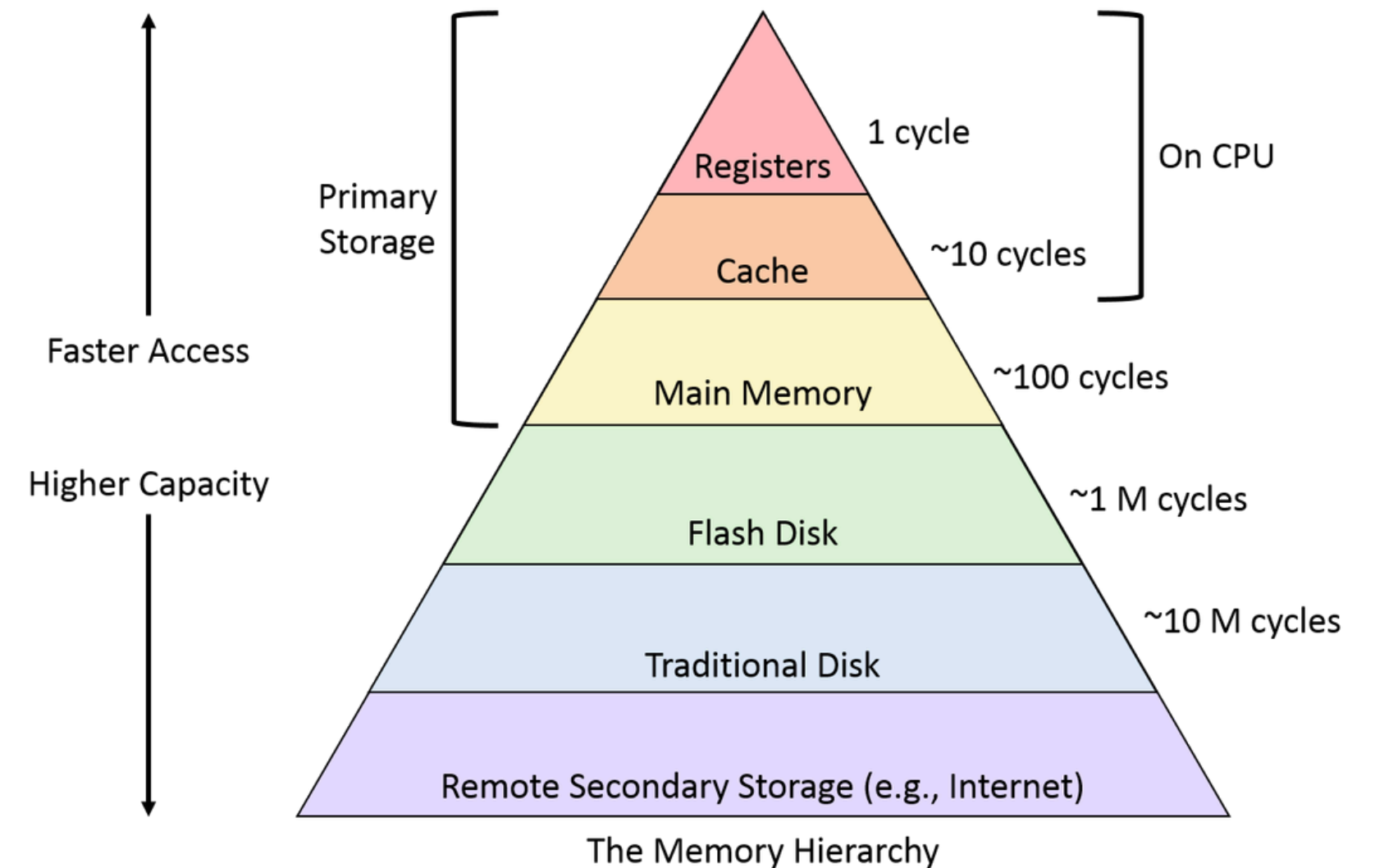
TTL

- Set expiration: ***EXPIRE*** <key> <seconds>
- Check remaining TTL: ***TTL*** <key>
- Remove expiration: ***PERSIST*** <key>

```
redis> SET mykey "Hello"  
OK  
redis> EXPIRE mykey 10  
(integer) 1  
redis> TTL mykey  
(integer) 10
```

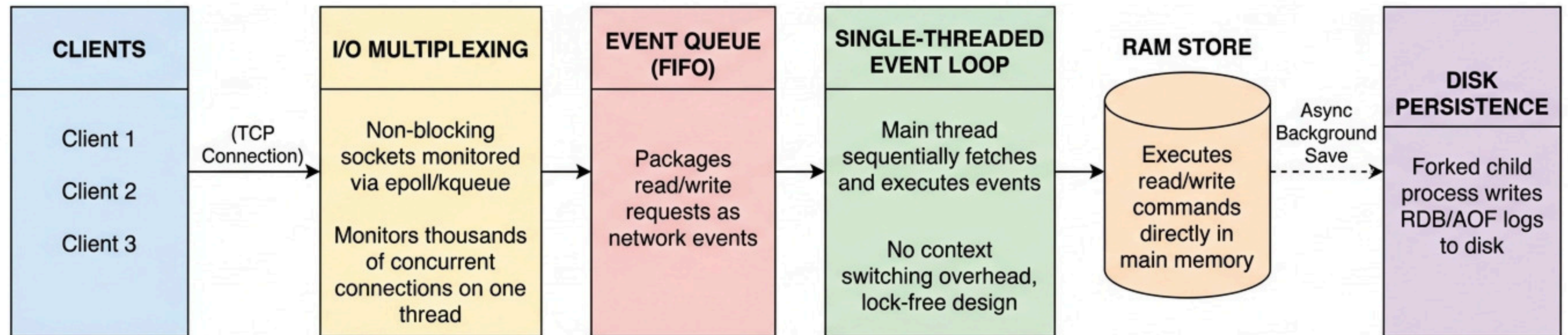

Memory Hierarchy

- Memory-first Architecture:
 - In-memory data storage
 - No disk I/O bottleneck
- Memory Hierarchy
- Latency:
 - RAM: 50–100 ns
 - SSD: 100–300 μ s
 - HDD: ~10 ms
- Redis: Optimized for low-latency, high-throughput workloads



Architecture

- Redis uses a single-threaded event loop with I/O multiplexing to maximize in-memory performance.



Configuration

- Max memory
- Expiration:
 - Passive: Remove expired keys on access
 - Active: Background scan (10 times/sec)
- Eviction Policy:
 - Key set: allkeys / volatile (all keys / TTL keys only)
 - Rule: LRU, LFU, Random, Least TTL, Noeviction
- Persistent:
 - Snapshot: save 60 100 (save a snapshot after 60s if at least 100 keys changed)
 - AOF: appendonly yes (log every write command)

Memcached

- Open-source distributed in-memory cache
- Optimized for high-performance caching
- Simple key-value data storage
- Supports client libraries for multiple programming languages



Memcached vs Redis

Features	Redis	Memcached
Data Types	Strings, Lists, Sets, Hashes, Streams,...	Simple key-value pairs
Persistence	Yes (RDB, AOF)	No (data lost after restart)
Performance	Very fast, feature-rich	Extremely fast, lightweight
Advanced Features	Pub/Sub, Transactions, Replication, Clustering	None
Best Use Case	Sessions, Leaderboards, Queues, Analytics	Simple caching, frequent lookups

Demo

Caching Concepts

- Storing an entire database in RAM is **too expensive** for most systems.
- Real-world access patterns follow two important principles:
 - **Temporal Locality**: Recently accessed data is likely to be accessed again in the near future.
 - **Zipfian Distribution (Pareto Principle)**: Around **80%** of requests target only **20%** of the data ("**hot keys**").
- **Key Insight**:
 - By caching only a small fraction (**20–30%**) of the dataset, we can serve **most requests** while significantly reducing database load.

Caching Strategy

When to use caching?

Good

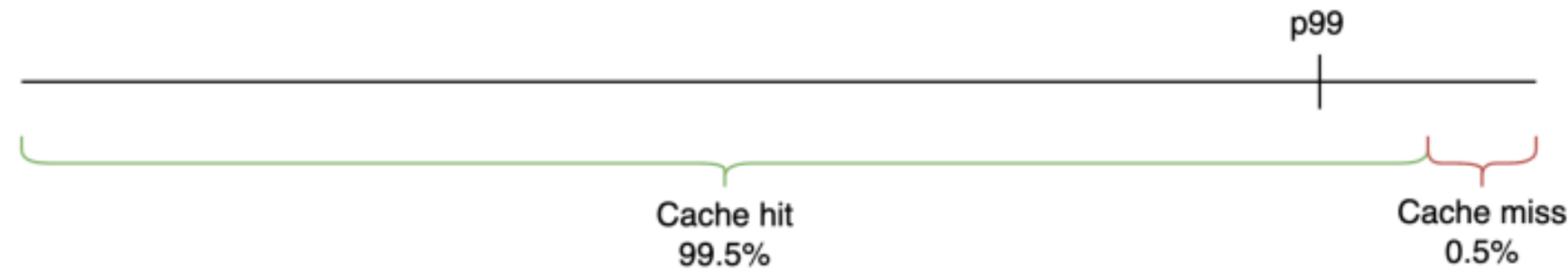
- **Read-heavy workloads** (social media, product catalogs)
- **Expensive queries** (aggregations, reports)
- **Rarely changing data** (configs, user profiles, static assets)

Bad

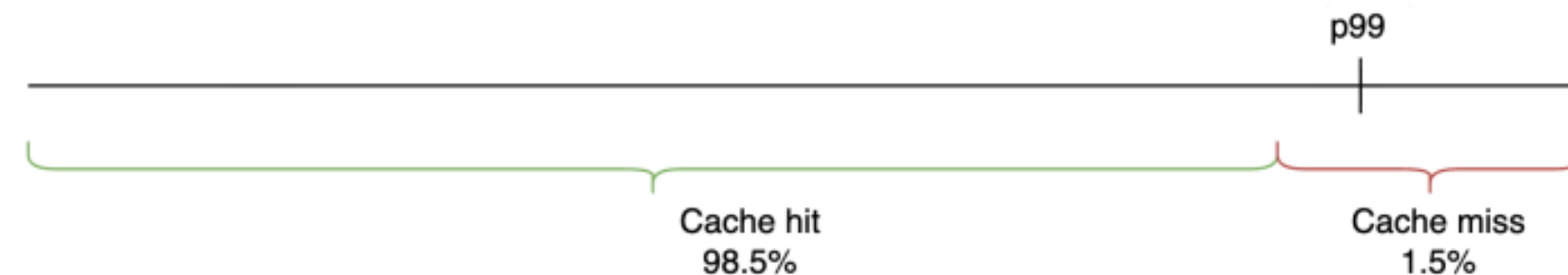
- **Write-heavy workloads** (frequent updates)
- **Strong consistency required** (banking, trading)
- **Fast & lightweight queries** (over-engineering)

Hit Rate and Tail Latency

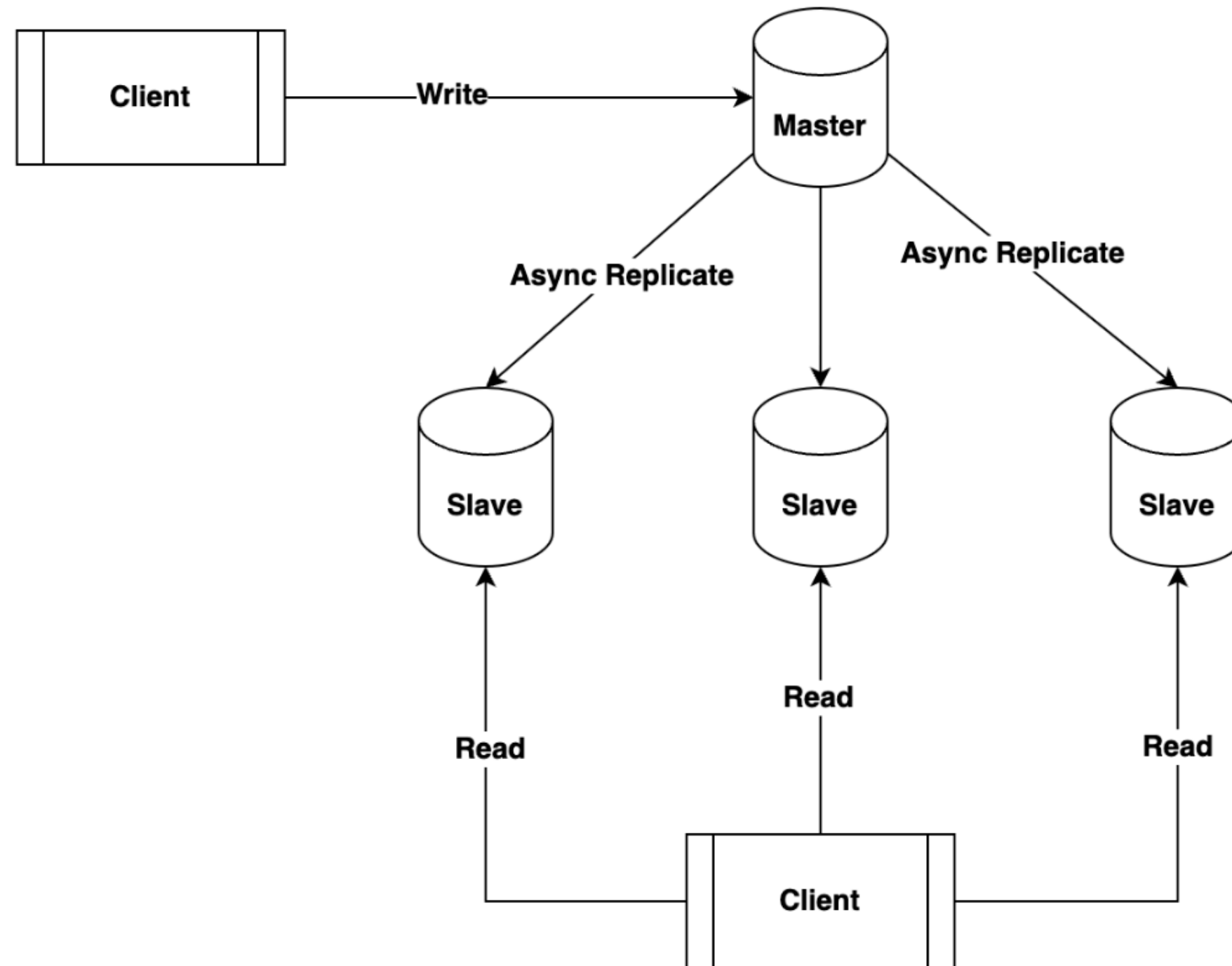
- **Tail latency (p99)** represents the slowest **1%** of requests in a system.
- **Hit Rate = 99.5%** (Miss Rate = 0.5%): Only **0.5%** of requests access the database, so p99 is still served from the cache (**~1 ms**).



- **Hit Rate = 98.5%** (Miss Rate = 1.5%): Since more than **1%** of requests miss the cache, p99 falls into database requests, increasing to **100+ ms**.



Master-Slave Replication



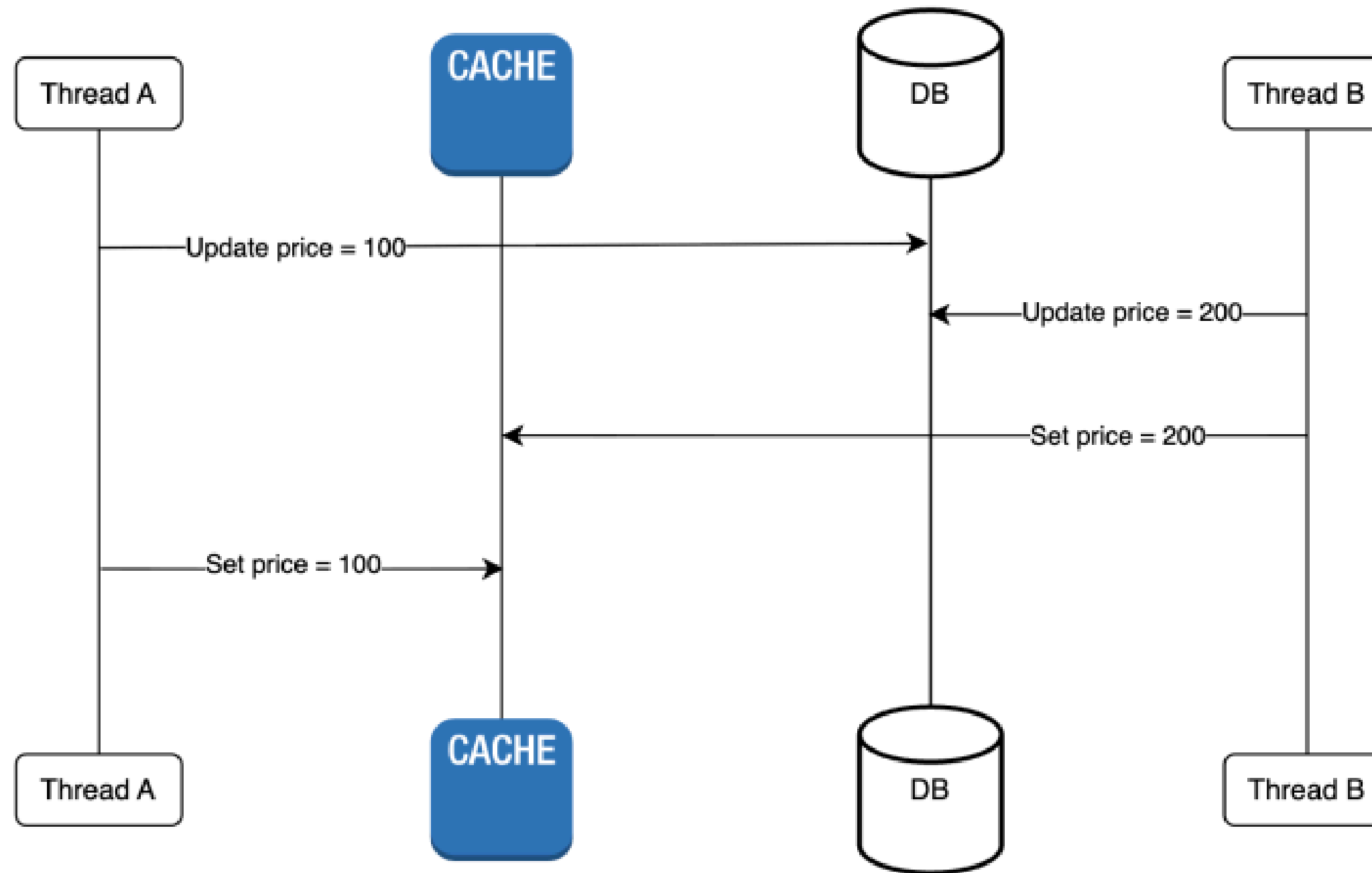
Cache Consistency

Cache Update Strategy: Update or Invalidate?

- When data in the database changes, there are two ways to update the cache:
 - **Update Cache:** *cache.set(key, newValue)* - Overwrite the cached value immediately.
 - **Invalidate Cache:** *cache.del(key)* - Remove the cache entry and let the next read fetch fresh data from the database.
- Prefer **Invalidate Cache**: It is the standard “Cache-Aside” Pattern

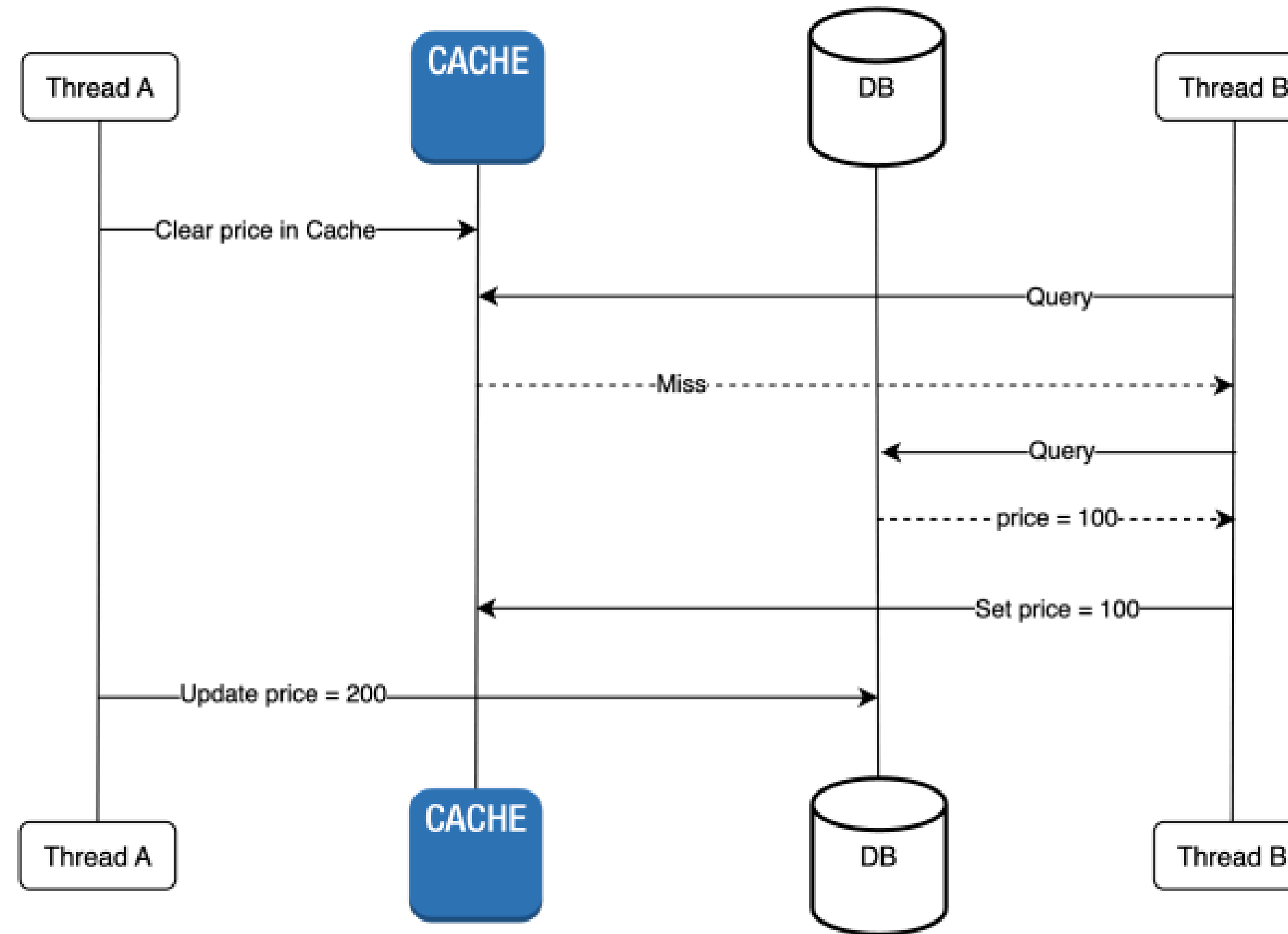
Cache Consistency

Why Not Update the Cache? (Double Write Problem)



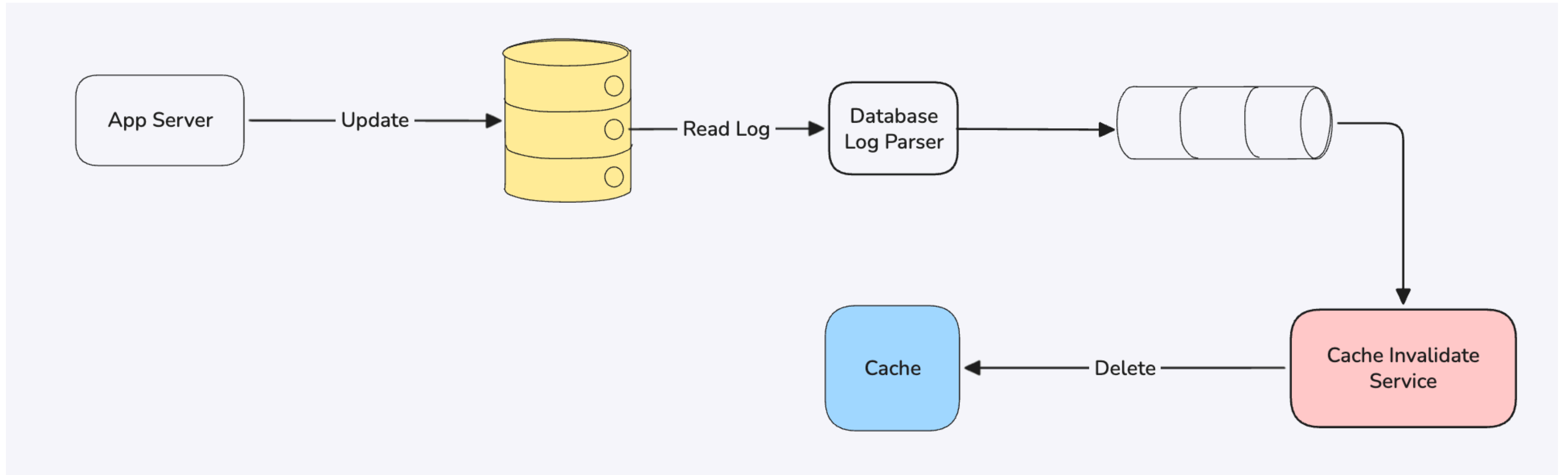
Cache Consistency

Operation Order: Invalidate Cache or Update the Database First?



Solution

Change Data Capture (CDC)



Solution

Lease & Remote Marker

- **Lease:** Grants exclusive permission to write to the cache.
- **Remote Marker:** Indicates that the data in the primary database has recently changed and that the replica database may not have synchronized the update yet.
- **Trade-offs** of Lease + Remote Marker
 - Addresses: Replication Lag + Zombie Readers
 - Complexity: More client-side logic
 - Latency: Higher P99 due to retries or primary reads
 - Marker Eviction: Markers may be evicted by LRU/LFU, reducing protection

Conclusion

*"There is no perfect cache consistency strategy,
only the one that best fits your system."*

